Rule-Based PCB Defect Detection Using Classical Image Processing

Amina Shamraze
City College of New York
160 Convent Ave, New York, NY, 10031
Ashamra001@citymail.cuny.edu

Abstract

This project presents a rule-based PCB defect detection pipeline for identifying open and short defects by comparing a reference PCB image with a test image. The method uses image alignment, binary differencing, morphological filtering, and local region analysis to detect and classify defects. Results on the current sample set improved from approximately 60% to 90%, demonstrating that classical image processing can be an effective and interpretable approach for PCB inspection

Introduction

Automated PCB inspection plays an important role in quality control because even small manufacturing defects, such as opens and shorts, can cause a board to fail electrically. A common image-based approach is to compare a defect-free reference image with a test image and look for regions where they differ. In practice, though, simple image subtraction is usually not enough. Slight misalignment, image noise, and very thin copper traces can create different regions that appear broken, shifted, or unclear, making it harder to tell the difference between a real defect and normal variation.

The goal of this project was to develop an interpretable, rule-based PCB defect detection pipeline using image processing techniques learned in class. The system was designed to detect and classify two common defect types: open circuits, where copper is missing, and the conductive path is broken, and short circuits, where extra copper creates an unintended connection. Unlike many public implementations that depend on machine learning, this project follows a classical image-processing approach so that each step of the detection logic remains visible, understandable, and easier to improve.

1. Project Objective

The objective was to design a system that compares a reference PCB image and a test PCB image, highlights regions that differ, and assigns each candidate region to a meaningful defect class. The pipeline needed to satisfy three practical requirements. First, it had to be robust to small image misalignment. Second, it had to suppress noise and insignificant speckles without removing real defects. Third, it had to distinguish between extra-copper and missing-copper behavior in a way that reflects the actual physical meaning of opens and shorts on a PCB.

2. Methodology

2.1 Binary Representation

The preprocessing stage converts images to greyscale f or further processing. After being converted to greyscale, image is then converted to binary image, so that copper traces and background can be separated clearly. A thresholding operation was used to map each pixel into one of two classes.

2.2 Alignment by translation search

This stage aligns the test image to the reference image so that corresponding traces and pads overlap as closely as possible. This step is necessary because even a small translation or rotation can create large false differences after subtraction.

To reduce false defect regions caused by small positional offsets, the test image was translated over a limited search window. The shift that produced the minimum XOR mismatch with the reference image was selected as the final alignment.

$$(dx^*, dy^*) = arg\min_{dx,dy} \sum_{x,y} (R(x,y) \oplus) T(x-dx, y-dy)) \ldots\ldots\ldots\ldots (1)$$

In equation (1), R (x,y) is the reference binary image, T(x,y) is the binary test image.

2.3. Logical defect extraction

After alignment, defect regions were extracted using logical operations on the binary reference and test images. An extra copper mask was computed as follows

$$M_{extra}(x,y) = R(x,y) \wedge \neg T(x,y) \ldots\ldots\ldots\ldots (2)$$

Similarly, missing copper-mask was computed using equation (3).

$$M_{missing}(x,y) = T(x,y) \wedge \neg R(x,y) \ldots\ldots\ldots\ldots (3)$$

This mask was used to identify pixels where copper appeared in the test image but not in the reference image. The overall defect mask was then defined using the XOR operation, 3.4. Filtering and morphological operations

To reduce thin edge artifacts and small spurious regions, the defect masks were refined using connected-component filtering and morphological operations. First, connected regions smaller than a minimum area threshold were removed:

$$A_{filtered} = \bigcup_{i:|C_i| \geq A_{min}} C_i \ldots\ldots\ldots\ldots (5)$$

Where Amin = 15 pixels for initial cleanup. Next, a morphological opening was applied.

Morphological operations were used to refine the raw defect masks after differencing. Opening helps remove small, isolated artifacts and thin protrusions, while closing helps fill small gaps and reconnect fragmented regions. These operations are useful in PCB inspection because raw difference masks often contain edge noise and broken defect regions caused by slight misalignment and thin traces.

$$A \circ S = (A \ominus S) \oplus S \ldots\ldots\ldots\ldots (6)$$

After that, morphological closing was applied;

$$A \bullet S = (A \oplus S) \ominus S \ldots\ldots\ldots\ldots (7)$$

Here, S denotes a disk-shaped structure element. The extra mask was less-aggressively processed to avoid removing weak close-circuit elements.

2.4. Region filtering

Even after morphological processing, some small or very thin regions may remain due to noise, imperfect alignment, or trace-edge artifacts. To reduce these false positives, each connected defect region was evaluated using simple geometric criteria. Each detected region was measured by area and thickness, and only regions large enough and thick enough to look physically meaningful were kept. This helped remove small residual fragments that survived the earlier morphological cleanup.

2.5. Debugging and Refinement

After the regions were filtered, the images were displayed. The detected defects were overlaid on the original images, and then regions were marked and labeled.

For this pipeline, the main issues were seen during the refinement stages. Early versions of the pipeline sometimes labeled missing-copper regions as shorts and extra-copper regions as opens because the logical mapping between binary masks and physical meaning was inconsistent. This issue was corrected by revisiting the copper/background convention and verifying how each subtraction mask should be interpreted.

A second major issue involved thin or noisy open-circuit traces. Some true opens were weakly represented in the defect mask, which caused them to be missed or marked as unclear. To address this, the defect-mask processing and classification flow were refined so that faint but structurally meaningful open regions could still be recognized. This refinement was the main reason for the observed performance improvement.

An important lesson from this debugging process was that pixel-level difference alone is not enough for reliable PCB defect classification. Correct interpretation depends on context: where the region lies, how it connects to nearby copper, and whether it changes electrical continuity

3. Results

On the current sample set, the earlier version of the system performed correctly on roughly 6 out of 10 cases. After correcting the open/short logic and improving the handling of weak open-circuit regions, the performance improved to approximately 9 out of 10 cases. This result indicates that the refined rule-based pipeline is substantially more reliable than the initial version.

Qualitatively, the final system produces cleaner defect masks, more meaningful candidate regions, and more accurate labels. The improvements are especially visible in cases where open-circuit defects appear as thin or noisy gaps in copper traces. These were the cases most likely to

be missed in earlier iterations.

Figure 1. Reference binary image

Figure 2. Aligned test binary image

Fig 3. Extra Copper Mask

Fig 4. Missing Copper

Fig 5. Filtered visible defects

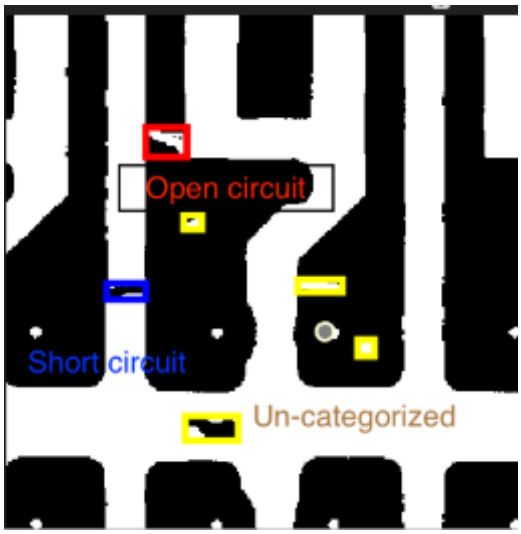
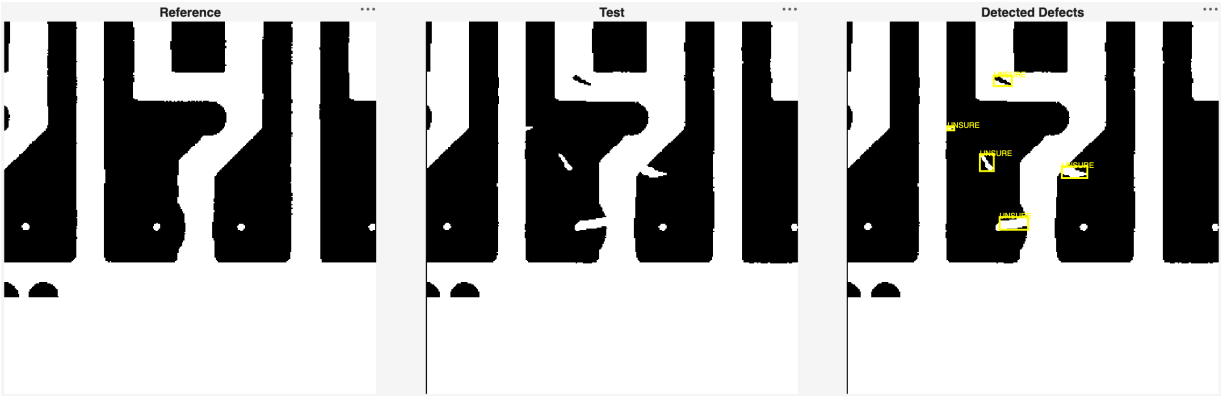
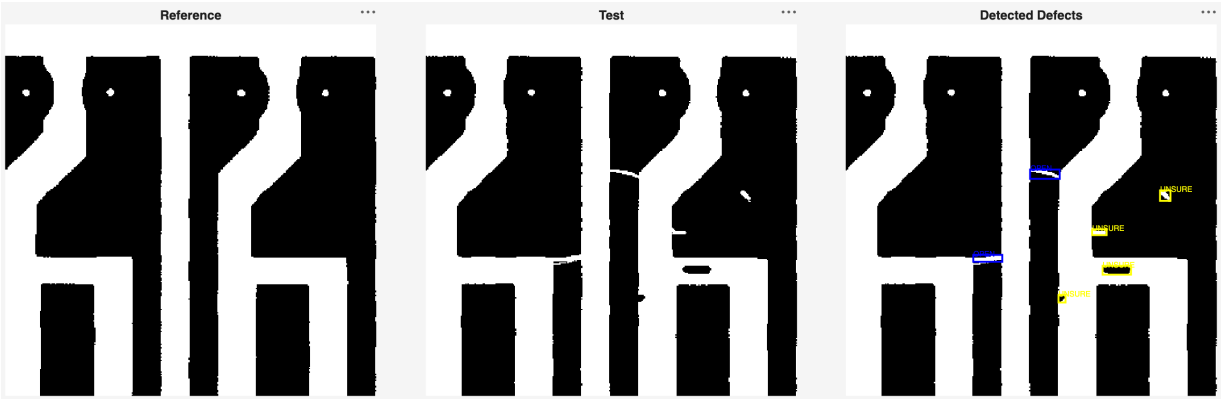
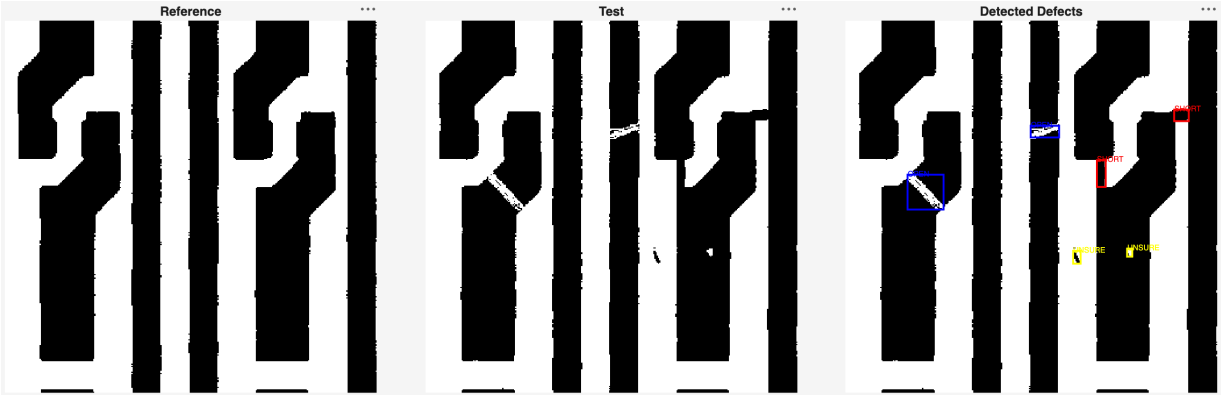


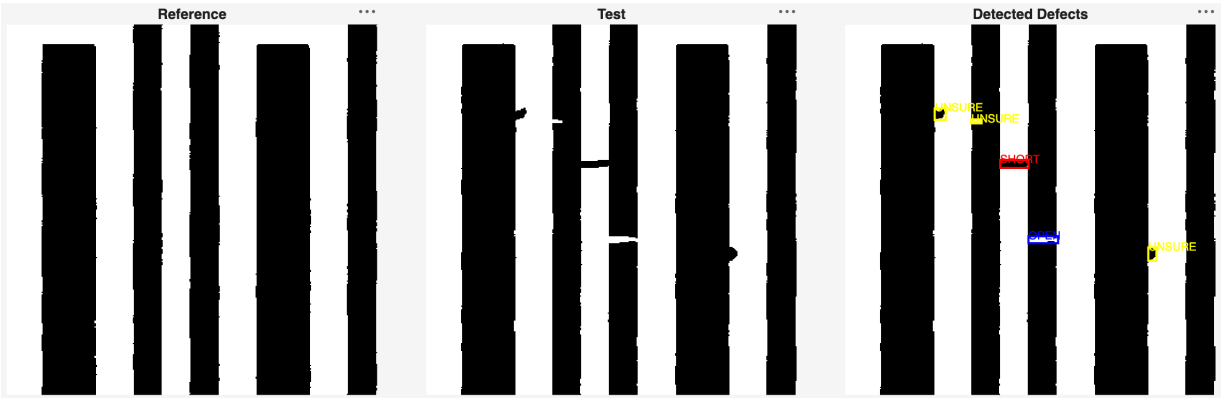
Fig 6. Final classified defects

Here, the detected defects are grouped into three categories: open, short, and uncategorized. The open label is assigned to defects in which a conductive trace appears visibly broken, indicating a possible interruption in current flow. The short label is assigned to regions where extra copper creates an unintended connection between nearby conductors, which may lead to electrical bridging. Defects that do not clearly fit either pattern, such as isolated copper fragments or holes appearing within copper or substrate regions, are placed in the uncategorized group. Although these regions are not ignored, they require further engineering judgment to determine whether they will affect circuit functionality. It should also be noted that the labels open and short are used here as practical visual categories for inspection; not every connected copper region necessarily produces a true short circuit, and not every broken pad or discontinuity necessarily results in a true open-circuit failure.


Reference
Test
Detected Defects
UNSURE
UNSURE
UNSURE
UNSURE
UNSURE


Reference
Test
Detected Defects
UNSURE
UNSURE
UNSURE


Reference
Test
Detected Defects
SHORT
SHORT
UNSURE
UNSURE


Reference
Test
Detected Defects
UNSURE
UNSURE
SHORT
UNSURE

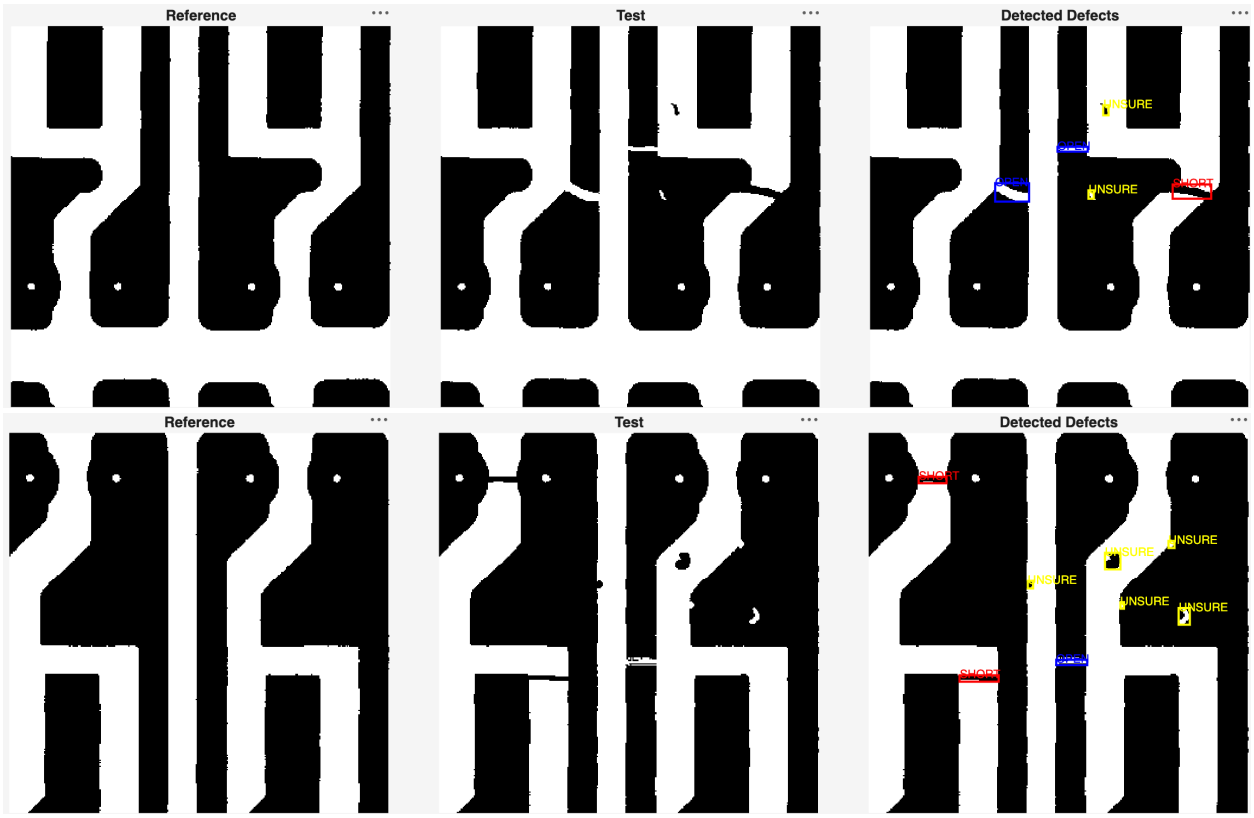


Fig. 7,8,9,10,11,12 show resulted images obtained after pipeline.

As it can be seen in images above, not every extra copper indicates the presence of short. Therefore region filtering played a very crucial role in categorizing the regions that were truly connecting the surrounding traces, and thus labelled them shorts. Likewise, region filtering proved to be useful for detecting open-circuits as well. Since the open-circuit cases have varying area, polarity and connectivity were used to make sure the resulting defects are labelled correctly.

Polarity and connectivity are used here because they separate what kind of pixel change happened from whether that change actually creates an electrical defect. Polarity tells the algorithm whether the detected region is mainly missing copper or extra copper. Since the binary convention is copper = 0 and background = 1, a missing-copper region means copper existed in the reference image but became background in the test image, while an extra-copper region means background in the reference became copper in the test image. However, polarity alone is not enough because not every missing-copper mark is a true open, and not every extra-copper mark is a true short. That is why the code also checks connectivity inside a local region around each defect. For an open circuit, the missing copper must break one originally connected trace into two or more disconnected copper components in the test image. For a short circuit, the extra copper must join two or more separate copper components from the reference into one connected component in the test image. This combination makes the classification stricter and more physically meaningful: polarity identifies whether the defect is missing or added copper, while connectivity verifies whether that change actually breaks or joins PCB traces.

The results observed are summarized as below;

Table 1. Image-level summary of detected defect regions.

Image ID	Total detected regions	Open	Short	Uncategorized
1	6	1	1	4
2	8	2	3	3
3	5	1	2	2
4	3	0	1	2
5	6	1	1	4
6	6	1	1	4
7	6	0	1	5
8	5	1	1	3
9	8	0	1	7
10	4	1	1	2
11	5	0	0	5
12	6	2	0	4
13	6	2	2	2
14	5	1	1	3
15	5	2	1	2
16	8	1	2	5
17	7	2	0	5
18	6	1	1	4
19	3	1	1	1
20	5	1	3	1

5. Discussions

This project demonstrates that classical image-processing techniques remain a strong and practical approach for PCB inspection when interpretability is important. The proposed pipeline is transparent, modular, and relatively easy to analyze because each stage performs a specific and understandable function. This makes the method well suited for an academic setting, where it is important not only to obtain a detection result but also to understand how preprocessing, differencing, and post-processing affect the final classification outcome.

At the same time, the method has several limitations. Its performance depends strongly on the quality of the source PCB images, including alignment accuracy, noise level, contrast, and the visibility of thin traces. Some defect regions remain difficult to isolate and classify reliably, particularly when the defects are weak, noisy, or visually ambiguous. In addition, the pipeline relies on manually tuned filtering and classification rules, which may not generalize as effectively to more complex board layouts or to a wider range of defect types.

The reported performance should therefore be interpreted in the context of the current sample set rather than as a production-level benchmark. Since the evaluation was performed on a limited set of PCB images rather than on a large, standardized dataset, the results represent strong preliminary performance under the chosen test conditions. Future improvements may include stronger continuity analysis, more robust topological reasoning, and evaluation on a larger and more diverse PCB image dataset.

6. Future Work

Several extensions can strengthen the project further. A larger and more diverse PCB image set should be used to evaluate generality. The open-circuit logic can be improved by incorporating explicit continuity checks along traces rather than relying only on local region shape. The short-circuit logic can also be refined using connectivity analysis to verify whether extra copper actually bridges two previously separate conductors. Finally, the classical pipeline can serve as a baseline for future comparison with machine-learning approaches.

7. Conclusion

A rule-based PCB defect detection system was developed to identify opens and shorts by comparing a reference PCB image against a test image. The final pipeline combines alignment, binary differencing, morphological cleanup, and local defect analysis to produce interpretable results. The most important contribution of the project was not only building the pipeline, but also debugging the logic so that the detected masks corresponded correctly to physical PCB defects. After refinement, the system improved to 90% performance on the current sample set, demonstrating that a carefully engineered image-processing workflow can achieve strong results without machine learning.

References

1. S. Tang, F. He, X. Huang, and J. Yang, “Online PCB defect detector on a new PCB defect dataset,” arXiv preprint arXiv:1902.06197, 2019.
2. R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Harlow, U.K.: Pearson, 2018.

3. P. Soille, Morphological Image Analysis: Principles and Applications, 2nd ed. Berlin, Germany: Springer, 2004.

4. MathWorks, “bwareaopen,” Image Processing Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/images/ref/bwareaopen.html

5. MathWorks, “imopen,” Image Processing Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/images/ref/imopen.html

6. MathWorks, “imclose,” Image Processing Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/images/ref/imclose.html